SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 4

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 4: JavaScript: first steps

Values, variables, maths

Today you write your first **JavaScript**, the language that makes a page *do* things. HTML says what things are, CSS says how they look, and JavaScript gives the browser instructions.

You work on the Friends page again. It's styled with the ShowShelf look (and your colours from Day 2), but some parts are empty: grey notes say what's missing. By the end of the day your JavaScript fills them in: the title, the facts, how long it takes to watch every episode, and a message that changes depending on the numbers.

Your code goes in a file called **app.js**. Keep an eye on the **Console** under the preview: it's where JavaScript talks back to you.

Go slowly. Every step is small on purpose, and every step saves automatically.

Step 1. Hello, Console ~10 min

Teaches: `console.log("...")` writes a message in the Console. Code runs from top to bottom.

TASK

1. Under the comments in the code box, write a `console.log` that says **Hello, ShowShelf!**
2. On the next line, write a second `console.log` with a sentence about a show you like, for example "My favourite show is Dark."

Look at the Console: two lines, in the same order as your code.

COMMON MISTAKES

- `Console.log` with a capital C, or `console.Log`.
- Forgetting the quotes: `console.log(Hello)`.
- Looking for the message on the page instead of in the Console.

NOTE

If the Console panel is hidden, click its title bar under the preview. Show students that the order of the lines in the Console follows the order in the code.

A CORRECT ANSWER

```
// app.js: the JavaScript for the Friends page.
// Lines that start with // are comments. JavaScript skips them.

console.log("Hello, ShowShelf!");
console.log("My favourite show is Dark.");
```

Step 2. Text and numbers ~15 min

Teaches: Strings are text in quotes, numbers have no quotes. `+` adds numbers but joins strings.

TASK

Under your two hello lines:

1. Log `236 + 62` (the episodes of Friends plus the episodes of Breaking Bad). No quotes.
2. Log `"236" + 62`, with quotes around the first number only. What do you get, and why?

COMMON MISTAKES

- Quotes around the whole sum: `console.log("236 + 62")` prints the text itself.
- A comma in decimals: `8,7` instead of `8.7`.

A CORRECT ANSWER

```
// ... unchanged ...
console.log("Hello, ShowShelf!");
console.log("My favourite show is Dark.");

console.log(236 + 62);
console.log("236" + 62);
```

Step 3. Variables with const ~15 min

Teaches: `const name = value;` creates a variable. Use its name without quotes.

TASK

Under your other lines, create three variables about Friends:

1. `showName` with the text `"Friends"`
2. `episodes` with the number `236`
3. `runtime` with the number `30` (minutes per episode)

Then log `showName` and log `episodes`.

COMMON MISTAKES

- Quotes around the variable when using it: `console.log("showName")`.
- Quotes around numbers: `const episodes = "236";`.
- A space in the name: `const show name`.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(236 + 62);
console.log("236" + 62);

const showName = "Friends";
const episodes = 236;
const runtime = 30;

console.log(showName);
console.log(episodes);
```

Step 4. Values that change: let ~15 min

Teaches: `let` for values that change. Change them with `name = new value;`.

TASK

1. Create `episodesWatched` with `let`, starting at `10`, and log it.
2. On a new line, add 1 to it. Then log it again. The Console should show 10, then 11.
3. Try this: change `let` to `const` and read the red error. Then change it back.

COMMON MISTAKES

- Writing `let` again on the second line: "has already been declared".
- Writing `episodesWatched + 1;` alone (it calculates 11 but doesn't store it).
- Using `const` and getting "Assignment to constant variable".

A CORRECT ANSWER

```
// ... unchanged ...
console.log(showName);
console.log(episodes);

let episodesWatched = 10;
console.log(episodesWatched);

episodesWatched = episodesWatched + 1;
console.log(episodesWatched);
```

Step 5. Maths with variables ~20 min

Teaches: Maths: `+ - * /`, results in variables. `Math.round()` rounds to a whole number.

TASK

Under your code, using your variables `episodes` and `runtime`:

1. Create `totalMinutes`: episodes times runtime. Log it.
2. Create `hours`: totalMinutes divided by 60. Log it.
3. Create `days`: hours divided by 24. Log it. (Lots of decimals!)
4. Log `Math.round(days)`.

COMMON MISTAKES

- `math.round` with a small m.
- `x` instead of `*` for times.
- Typing the numbers (`236 * 30`) instead of the variable names.

A CORRECT ANSWER

```
// ... unchanged ...
episodesWatched = episodesWatched + 1;
console.log(episodesWatched);

const totalMinutes = episodes * runtime;
console.log(totalMinutes);

const hours = totalMinutes / 60;
console.log(hours);

const days = hours / 24;
console.log(days);
console.log(Math.round(days));
```

Step 6. Sentences with backticks ~15 min

Teaches: Template literals: text between backticks, with `${variable}` for the values.

TASK

Create a variable `sentence` with a template literal that makes this text from your variables:

Watching all 236 episodes of Friends takes 118 hours.

Use `${episodes}`, `${showName}` and `${hours}` for the parts that come from variables. Then log `sentence`.

COMMON MISTAKES

- Normal quotes instead of backticks: the `${ }` is printed as it is.
- `$(hours)` or `{hours}` instead of `${hours}`.
- Forgetting spaces around the `${ }` parts, so words stick together.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(days);
console.log(Math.round(days));

const sentence = `Watching all ${episodes} episodes of ${showName} takes ${hours} hours.`;
console.log(sentence);
```

Step 7. Put text on the page ~20 min

Teaches: `document.querySelector("#id").textContent = value;` puts text on the page.

TASK

At the bottom of your code, fill three elements:

1. `#title` gets `showName`
2. `#meta` gets a template literal: **236 episodes · 30 minutes each** (use `${episodes}` and `${runtime}`)
3. `#watch-time` gets your `sentence`

Watch the grey notes disappear.

COMMON MISTAKES

- Forgetting the `#` in the selector: `querySelector("title")` changes the browser tab's title instead.
- `textcontent` with a small c: no error, but nothing happens.
- Quotes around the variable: the page shows the word *showName*.

A CORRECT ANSWER

```
// ... unchanged ...
const sentence = `Watching all ${episodes} episodes of ${showName} takes ${hours} hours.`;
console.log(sentence);

document.querySelector("#title").textContent = showName;
document.querySelector("#meta").textContent = `${episodes} episodes · ${runtime} minutes each`;
document.querySelector("#watch-time").textContent = sentence;
```

Step 8. True or false ~15 min

Teaches: Booleans are `true` or `false`. Compare with `> < >= <= === !==`.

TASK

1. Create `isLong`: whether `episodes` is greater than 100. Log it.
2. Log `showName === "Friends"`.
3. Log `runtime === "30"`. Why is the answer `false`?

COMMON MISTAKES

- One `=` in a comparison: `showName = "Friends"` tries to change the variable.
- Quotes around `true`: `"true"` is a string, not a boolean.

A CORRECT ANSWER

```
// ... unchanged ...
document.querySelector("#meta").textContent = `${episodes} episodes · ${runtime} minutes each`;
document.querySelector("#watch-time").textContent = sentence;

const isLong = episodes > 100;
console.log(isLong);
console.log(showName === "Friends");
console.log(runtime === "30");
```

Step 9. Choose with if and else ~20 min

Teaches: `if (condition) { ... } else { ... }` runs one of two blocks.

TASK

At the bottom of your code, write an `if / else`:

- If the show is long (`isLong`), put **A long one: get comfortable on the sofa!** in `#verdict`.
- Else, put **A short one: perfect for a weekend.** in `#verdict`.

Then change `episodes` to `10` at the top and watch the message change. Change it back to `236` when you're done.

COMMON MISTAKES

- A semicolon after the condition: `if (isLong); {` (the block then always runs).
- Missing `}` before `else`.
- Putting both messages outside the blocks, so the second one always wins.

NOTE

The check runs the student's code once more with `episodes` set to 5 to test the `else` block, so a hard-coded condition like `if (true)` won't pass.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(showName === "Friends");
console.log(runtime === "30");

if (isLong) {
  document.querySelector("#verdict").textContent = "A long one: get comfortable on the sofa!";
} else {
  document.querySelector("#verdict").textContent = "A short one: perfect for a weekend.";
}
```

Step 10. Read the error, fix the code ~15 min

Teaches: Red errors name the problem and the line. A syntax error stops the whole file.

TASK

We added two broken lines at the bottom of your code. Look at the page: the grey notes are back, because one mistake stopped the whole file.

1. Read the red error and its line number. Fix that mistake.
2. A different error appears. Read it, and fix that one too.

When both are fixed, the badge above the title says **NBC**.

COMMON MISTAKES

- Fixing the line number in the message but not looking at the line above.
- Deleting the broken lines instead of fixing them.

NOTE

Encourage students to read the error out loud. The grey explanation under each error is written for beginners.

A CORRECT ANSWER

```
// ... unchanged ...
  document.querySelector("#verdict").textContent = "A short one: perfect for a weekend.";
}

// Step 10: two mistakes are hiding below. The Console will help you find them.
const network = "NBC";
document.querySelector("#network").textContent = network;
```

Extra 1. Text that knows tricks ~10 min

Teaches: `.length` counts characters; `.toUpperCase()` and `.toLowerCase()` change the case.

TASK

1. Log the number of characters in `showName`.
2. At the very bottom, set `#title` to `showName` in capital letters.

A CORRECT ANSWER

```
// ... unchanged ...
const network = "NBC";
document.querySelector("#network").textContent = network;

console.log(showName.length);
document.querySelector("#title").textContent = showName.toUpperCase();
```

Extra 2. A random episode ~15 min

Teaches: `Math.floor(Math.random() * n) + 1` is a random whole number from 1 to n.

TASK

1. Create `randomEpisode`: a random whole number from 1 to `episodes`.
2. Put **Tonight's random pick: episode 137** (with your random number) in `#random`.

Press **Run again** above the preview a few times: the number changes.

A CORRECT ANSWER

```
// ... unchanged ...
const network = "NBC";
document.querySelector("#network").textContent = network;

const randomEpisode = Math.floor(Math.random() * episodes) + 1;
document.querySelector("#random").textContent = `Tonight's random pick: episode ${randomEpisode}`;
```

Extra 3. Three choices with else if ~15 min

Teaches: `if / else if / else` chooses between several cases, top to bottom.

TASK

1. Create `const rating = 8.8;`.
2. Show a label in `#rating`: ★ **Excellent** if rating is 9 or more, ★ **Very good** if it's 8 or more, and ★ **Worth a try** otherwise.

Try changing the rating to 9.5 and to 7 to see all three labels.

A CORRECT ANSWER

```
// ... unchanged ...
const network = "NBC";
document.querySelector("#network").textContent = network;

const rating = 8.8;

if (rating >= 9) {
  document.querySelector("#rating").textContent = "★ Excellent";
} else if (rating >= 8) {
  document.querySelector("#rating").textContent = "★ Very good";
} else {
  document.querySelector("#rating").textContent = "★ Worth a try";
}
```